Apartment Portfolio Database Design Report

Group 2: Linyu Zhao, Yuhan Luan, Xin Cai, Xinru Tang

1. Business/Organization Description

1.1 Overview

This project designs a database for a residential property management company that operates a portfolio of multiple apartment communities rather than a single building. Each property in the portfolio is overseen by an assigned manager and contains a number of rentable units, which are leased to tenants under individually tracked lease agreements. Beyond core leasing, the company also manages three supporting operations that are common to multi-property residential portfolios: on-site parking, maintenance and inspection services, and a mixed workforce of in-house and contractor technicians who carry out that work. The scope of the system covers the full operational lifecycle of a property: onboarding a new property and its units, signing and renewing leases, collecting rent, assigning parking, and dispatching maintenance, across as many properties as the company adds to its portfolio.

1.2 Purpose and Benefits

Property management companies of this size typically run each of these functions, including leasing, parking, maintenance, and technician scheduling, on separate spreadsheets or paper logs kept per property. This makes it difficult to answer basic cross-portfolio questions, such as which units are vacant this month, which leases expire soon, or which technician is overloaded, and it creates real data-integrity risks: a manager's contact information gets copied into every property record and drifts out of sync, a parking spot's current holder is overwritten and its assignment history is lost, and a maintenance request can sit unassigned with no reliable way to flag it as overdue. A centralized relational database addresses these problems directly by storing each fact, such as a lease, a payment, a parking assignment, or a maintenance request, exactly once, connected by keys rather than duplicated across files. This gives the company a single, consistent source of truth across every property it manages, and lets day-to-day staff, including leasing agents,

accounting, and maintenance coordinators, as well as administrators, work from the same underlying data instead of reconciling separate records after the fact.

1.3 Anticipated Functionality and Features

Built on the relational model described in the Logical Design section, the database is expected to support:

- **Portfolio and manager oversight:** tracking every property, its address, and year built, and the manager accountable for it, so a manager's information is maintained in one place even when they oversee several properties.
- **Full lease lifecycle management:** creating, renewing (via a self-referencing link between a lease and the one it replaces), and tracking the status of every lease, along with its associated payment history.
- **Unit and occupancy tracking:** maintaining each unit's physical details, including bedrooms, square footage, base rent, and operational status, and computing current occupancy from active leases rather than a manually updated flag.
- **Tenant relationship records:** centralized contact, employment, and move-in information for every tenant the company has leased to.
- **Parking operations:** an inventory of parking spots per property, with a full history of tenant assignments so availability can be derived rather than tracked by hand.
- **Maintenance and inspection workflow:** logging requests from a tenant or from staff, tracking them from open to completed, and recording scheduled and completed inspections.
- **Technician workforce management:** a shared record for every technician, whether in-house staff or an outside contractor, with the role-specific details each type requires, such as hourly wage versus billing rate and department versus contract terms, plus the skills each technician holds.

- **Reporting for day-to-day decision-making:** because every operational record lives in one connected schema, the company can generate reports on occupancy and vacancy rates, rent-roll and collections, parking utilization, maintenance turnaround time, and technician workload. This kind of cross-property visibility is something loose spreadsheets cannot reliably provide.

2. Conceptual Design: EER Model

2.1 Data Elements and Business Processes

Running a multi-property apartment portfolio requires tracking data across four connected business processes: leasing, parking, maintenance and inspection, and technician scheduling. Each process depends on a distinct set of data elements. Leasing needs a record of every property, its rentable units, the tenants who occupy them, the lease agreements that govern occupancy, and the payments collected against those leases. Parking needs an inventory of parking spots and a history of which tenant held which spot over time. Maintenance and inspection need a record of every request and inspection, whether or not a technician has been assigned yet, and who reported the issue. Technician scheduling needs a roster of the workforce that carries out maintenance and inspections, distinguishing in-house staff from outside contractors because each type has different attributes to track. These four processes, and the data elements each one needs, are what the EER model in this section is built to represent.

2.2 EER Model

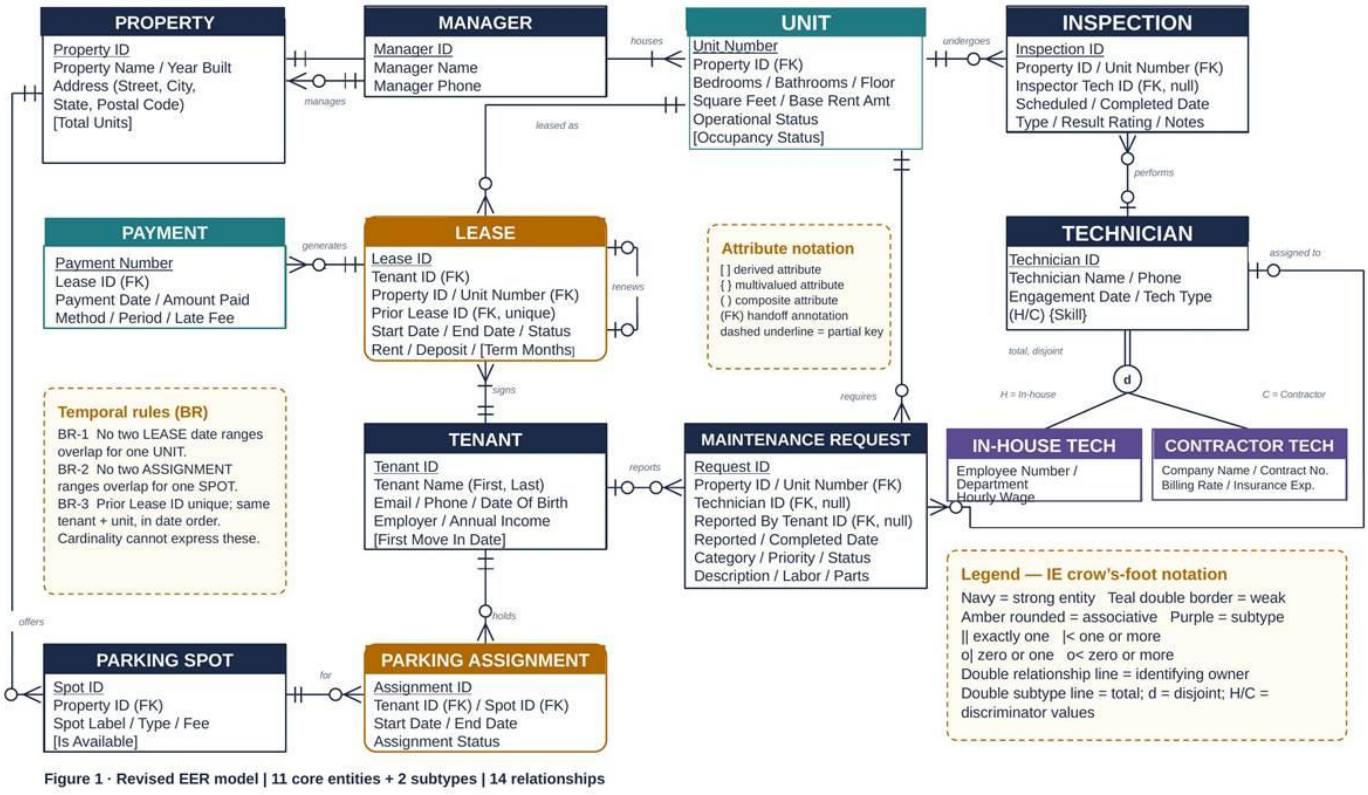


Figure 1. Revised EER model, 11 core entities plus 2 subtypes, 14 relationships.

The model uses IE crow's-foot notation. A double bar means exactly one, a bar with a crow's foot means one or more, a circle with a single bar means zero or one, and a circle with a crow's foot means zero or more. A double relationship line marks an identifying relationship, meaning the entity on the far side is a weak entity that depends on the entity it connects to for its own primary key. Square brackets mark a derived attribute that is computed rather than stored, curly braces mark a multivalued attribute, and parentheses mark a composite attribute made of smaller atomic parts. UNIT and PAYMENT are weak entities, shown with a teal double border, because their identifiers only make sense in the context of their owning entity. LEASE and PARKING ASSIGNMENT are associative entities, shown in amber, because each one resolves a many-to-many relationship (TENANT to UNIT, and TENANT to PARKING SPOT) while also carrying its own attributes and history. TECHNICIAN is a supertype with two subtypes, IN-HOUSE TECH and CONTRACTOR TECH, shown in purple, and the double line with the letter d marks this specialization as total and disjoint: every technician belongs to exactly one of the two subtypes, never both and never neither.

2.3 Entities and Attributes

The model has 13 entity types in total: 11 core entities plus the 2 subtypes of TECHNICIAN. Table 1 summarizes each entity, its category (strong, weak, associative, or supertype/subtype), its primary key, its key attributes, and a short note on its role. The complete attribute list for every entity, including derived and multivalued attributes, is shown in Figure 1.

Entity	Category	Primary Key	Selected Attributes	Notes
PROPERTY	Strong	Property ID	Property Name, Year Built, Address (Street, City, State, Postal Code), [Total Units] (derived)	Top-level record for each apartment community in the portfolio.
MANAGER	Strong	Manager ID	Manager Name, Manager Phone	Kept separate from PROPERTY so one manager's details are stored once even when they oversee several properties.
UNIT	Weak	Property ID/ Unit Number	Bedrooms, Bathrooms, Floor, Square Feet, Base Rent Amt, Operational Status, [Occupancy Status] (derived)	Exists only in the context of its owning PROPERTY.
PAYMENT	Weak	Lease ID/ Payment Number	Payment Date, Amount Paid, Payment Method, Period Covered, Late Fee Amt	Exists only in the context of its owning LEASE.
LEASE	Associative	Lease ID	Tenant ID (FK), Property ID/Unit Number (FK), Prior Lease ID (FK, self-referencing, nullable), Start/End Date, Status, Rent, Deposit, [Term Months] (derived)	Links TENANT and UNIT and carries its own attributes; the self-referencing FK chains a renewal to the lease it replaces.

TENANT	Strong	Tenant ID	Tenant Name (First, Last), Email, Phone, Date of Birth, Employer, Annual Income, [First Move In Date] (derived)	Under the single-tenant-per-lease scope, each tenant record represents one named lease holder.
PARKING SPOT	Strong	Spot ID	Property ID (FK), Spot Label, Spot Type, Monthly Fee, [Is Available] (derived)	Physical inventory of parking spaces per property.
PARKING ASSIGNMENT	Associative	Assignment ID	Tenant ID (FK), Spot ID (FK), Start/End Date, Assignment Status	Replaces a direct TENANT to PARKING SPOT link so reassignment history is preserved.
MAINTENANCE REQUEST	Strong	Request ID	Property ID/Unit Number (FK), Technician ID (FK, nullable), Reported By Tenant ID (FK, nullable), Reported/Completed Date, Category, Priority, Status, Description, Labor Hours, Parts Cost	May be logged by staff instead of a tenant, and may exist unassigned before dispatch.
INSPECTION	Strong	Inspection ID	Property ID/Unit Number (FK), Inspector Tech ID (FK, nullable), Scheduled/Completed Date, Type, Result Rating, Notes	May be scheduled before an inspector is assigned.
TECHNICIAN	Supertype	Technician ID	Technician Name, Phone, Engagement Date, Tech Type (H/C, discriminator), {Skill} (multivalued)	Shared record for both in-house staff and outside contractors.
IN-HOUSE TECH	Subtype of TECHNICIAN	Technician ID (FK)	Employee Number, Department, Hourly Wage	Present only for technicians with Tech Type = H.
CONTRACTOR TECH	Subtype of TECHNICIAN	Technician ID (FK)	Company Name, Contract Number, Billing Rate, Insurance Expiry	Present only for technicians with Tech Type = C.

Table 1. Entity summary.

2.4 Relationships and Cardinality

The model defines 14 named relationships, listed in Table 2, plus one specialization relationship connecting TECHNICIAN to its two subtypes. Cardinality is read from the far end of each relationship: the symbol next to an entity describes how many instances of that entity correspond to one instance of the entity at the other end.

Relationship	Entities Connected	Cardinality	Meaning
manages	MANAGER to PROPERTY	1 to zero-or-many	One manager may oversee several properties; every property has exactly one accountable manager.
houses	PROPERTY to UNIT	1 to zero-or-many (identifying)	Every unit belongs to exactly one property; a property may contain many units.
offers	PROPERTY to PARKING SPOT	1 to zero-or-many	Every parking spot belongs to exactly one property.
leased as	UNIT to LEASE	1 to zero-or-many	A unit may be leased many times over its history, never concurrently (BR-1).
generates	LEASE to PAYMENT	1 to zero-or-many (identifying)	Every payment belongs to exactly one lease.
signs	TENANT to LEASE	1 to one-or-many	Under the single-tenant-per-lease scope, every lease has exactly one tenant; every tenant record also implies at least one lease, since a tenant only exists in the system once they've signed.
renews	LEASE to LEASE (self-referencing)	zero-or-one to zero-or-one	Links a lease to the earlier lease it replaces, when one exists (BR-3).
undergoes	UNIT to INSPECTION	1 to zero-or-many	Every inspection targets exactly one unit.
requires	UNIT to MAINTENANCE REQUEST	1 to zero-or-many	Every maintenance request targets exactly one unit.
reports	TENANT to MAINTENANCE REQUEST	zero-or-one to zero-or-many	A request may be logged by staff instead of a tenant, so this side is optional.

assigned to	TECHNICIAN to MAINTENANCE REQUEST	zero-or-one to zero-or-many	A request may exist unassigned before dispatch.
performs	TECHNICIAN to INSPECTION	zero-or-one to zero-or-many	An inspection may be scheduled before a technician is assigned.
holds	TENANT to PARKING ASSIGNMENT	1 to zero-or-many	Every assignment record belongs to exactly one tenant.
for	PARKING SPOT to PARKING ASSIGNMENT	1 to zero-or-many	A spot can carry zero, one, or many assignment records across its history.

Table 2. Relationship summary.

2.5 Business Rules Beyond Cardinality

Three business rules govern the timing of records in this model and cannot be expressed through cardinality alone, because cardinality only constrains how many rows relate to each other, not how their date ranges behave over time. BR-1 states that no two LEASE date ranges may overlap for the same UNIT, so a unit cannot be leased to two tenants at once. BR-2 states that no two PARKING ASSIGNMENT date ranges may overlap for the same PARKING SPOT, for the same reason. BR-3 states that Prior Lease ID, when present, must reference an earlier lease for the same tenant and the same unit, with a start date that follows the prior lease's end date, so a renewal chain stays consistent. All three rules are enforced at the application or transaction level, as described in the Implementation and Application section, rather than by the schema itself.

2.6 Confirmed Scope Assumptions

The team confirmed four scope decisions while finalizing this model, each of which is worth stating explicitly because it shapes what the database can and cannot represent.

- **One tenant per lease:** the model adopts single-tenant leases, matching the original proposal. Co-tenants are out of scope; supporting them would require a LEASE_TENANT junction table and would change the current 14-table logical design.

- **MANAGER is its own entity:** manager details were moved off PROPERTY into a separate MANAGER entity, so a manager who oversees several properties has their information stored once rather than repeated on every property record.
- **Unit-level work only:** maintenance requests and inspections are attached to a UNIT, so common-area work such as a lobby, roof, or garage has no home in the current model. This is acceptable for the current scope; a nullable property-level link would extend it.
- **Tenants must hold a lease:** the relationship between TENANT and LEASE is mandatory on both sides, so a tenant record exists only once a first lease is signed. Prospective applicants are out of scope under this design.

3. Logical Design: Transformation to Relational Model

This section transforms the EER model presented in Section 2 into a relational schema in third normal form. Four mapping rules, applied consistently across every entity and relationship in the model, produced 14 tables, within the 10 to 20 table range required by the project guidelines. No table count was targeted directly; 14 is simply the result of applying these rules to the model in Figure 1.

3.1 Mapping Rules Applied

Rule	EER Construct	Relational Mapping	Tables
1	Weak entities	Composite primary key: the owner's key plus a local key	UNIT, PAYMENT
2	Associative entities	Own primary key, surrogate or composite	LEASE, PARKING_ASSIGNMENT
3	Multivalued attributes	Separate table with a composite primary key	TECHNICIAN_SKILL
4	Supertype/subtype specialization	Discriminator-driven split into a supertype table plus one table per subtype	TECHNICIAN, IN_HOUSE_TECH, CONTRACTOR_TECH

Table 1. EER to relational mapping rules.

For Rule 4, separate subtype tables were chosen over a single table with a discriminator column and nullable subtype-specific fields. Every column in IN_HOUSE_TECH and CONTRACTOR_TECH is meaningful for every row in that table, and neither table carries columns that only make sense for the other technician type, which a single merged table would otherwise require as structural NULLs.

3.2 Relational Schema

Table 2 presents the complete relational schema. An asterisk marks a primary key attribute, and a dagger marks a foreign key attribute; a composite primary key has more than one asterisked attribute. UNIT and PAYMENT are weak entities, so their primary key is the owning entity's key combined with a local key. LEASE and PARKING_ASSIGNMENT are associative entities with their own surrogate primary key. Every foreign key to UNIT carries both PropertyID and UnitNumber, since that pair is UNIT's full composite primary key.

Relation	Attributes (* = primary key, † = foreign key)
MANAGER	ManagerID*, ManagerName, ManagerPhone
PROPERTY	PropertyID*, PropertyName, Street, City, State, PostalCode, YearBuilt, ManagerID†
UNIT	PropertyID*†, UnitNumber*, Bedrooms, Bathrooms, Floor, SquareFeet, BaseRentAmt, OperationalStatus
PAYMENT	LeaseID*†, PaymentNumber*, PaymentDate, AmountPaid, PaymentMethod, PeriodCovered, LateFeeAmt
LEASE	LeaseID*, TenantID†, PropertyID†, UnitNumber†, PriorLeaseID†, StartDate, EndDate, LeaseStatus, MonthlyRent, DepositAmt
TENANT	TenantID*, FirstName, LastName, Email, Phone, DateOfBirth, Employer, AnnualIncome
PARKING_SPOT	SpotID*, PropertyID†, SpotLabel, SpotType, MonthlyFee
PARKING_ASSIGNMENT	AssignmentID*, TenantID†, SpotID†, StartDate, EndDate, AssignmentStatus
MAINTENANCE_REQUEST	RequestID*, PropertyID†, UnitNumber†, TechnicianID†, ReportedByTenantID†, ReportedDate, CompletedDate, Category, Priority, RequestStatus, Description, LaborHours, PartsCost

INSPECTION	InspectionID*, PropertyID†, UnitNumber†, InspectorTechID†, ScheduledDate, CompletedDate, InspectionType, ResultRating, Notes
TECHNICIAN	TechnicianID*, TechnicianName, Phone, EngagementDate, TechType
IN_HOUSE_TECH	TechnicianID*†, EmployeeNumber, Department, HourlyWage
CONTRACTOR_TECH	TechnicianID*†, CompanyName, ContractNumber, BillingRate, InsuranceExpiry
TECHNICIAN_SKILL	TechnicianID*†, Skill*

Table 2. Relational schema, short-text notation.

3.3 Normalization Steps

The schema in Table 2 was verified against first, second, and third normal form as follows.

First normal form: every attribute holds a single atomic value, and no table has a repeating group. PROPERTY.Address, which was a composite attribute on the EER diagram, was decomposed into Street, City, State, and PostalCode. TENANT. Tenant Name was decomposed into FirstName and LastName. TECHNICIAN's multivalued Skill attribute was pulled into its own table, TECHNICIAN_SKILL(TechnicianID, Skill), rather than left as a repeating value on TECHNICIAN. The derived attributes shown in brackets on the EER diagram, such as [Total Units], [Occupancy Status], [Term Months], [First Move In Date], and [Is Available], are computed values rather than stored columns and are excluded from the schema for this reason.

Second normal form: only needs to be checked on the three tables with a composite primary key: UNIT (PropertyID, UnitNumber), PAYMENT (LeaseID, PaymentNumber), and TECHNICIAN_SKILL (TechnicianID, Skill). In UNIT, Bedrooms, Bathrooms, Floor, SquareFeet, BaseRentAmt, and OperationalStatus all depend on the full composite key, not on either column alone, so no partial dependency exists. In PAYMENT：PaymentDate, AmountPaid, PaymentMethod, PeriodCovered, and LateFeeAmt all depend on the full (LeaseID, PaymentNumber) pair. TECHNICIAN_SKILL has no non-key attribute beyond its two-column key, so it trivially satisfies the second normal form. Every other table uses a single-column primary key and automatically satisfies the second normal form.

Third normal form: each table was checked for non-key attributes that depend on another non-key attribute rather than directly on the primary key.

LEASE, MAINTENANCE_REQUEST, and INSPECTION each carry both PropertyID and UnitNumber, which resembles a transitive dependency at first glance, since PropertyID looks like it could be derived through UnitNumber to UNIT to PropertyID. This is not a violation: because UNIT's primary key is the composite (PropertyID, UnitNumber), any table referencing UNIT must carry both columns to form a valid foreign key, and the same UnitNumber value recurs across different properties, so UnitNumber alone does not determine PropertyID. While mapping PROPERTY to its relational form, this transitive dependency was checked directly: had Manager Name and Manager Phone been carried over onto PROPERTY, they would have repeated across every property overseen by the same manager, since both attributes depend on the manager rather than on PropertyID. This is exactly why MANAGER was separated out as its own entity during conceptual design (Section 2.6); the relational mapping confirms that decision holds up under 3NF and carries the entity forward as-is, with PROPERTY.ManagerID as the connecting foreign key rather than storing manager details directly.

A similar dependency was checked on CONTRACTOR_TECH: BillingRate could in principle depend on CompanyName rather than on TechnicianID directly, if a contracting company charges a single flat rate for every technician it sends. This schema assumes billing rate is negotiated per technician rather than per company, since contractors from the same firm may be billed differently based on experience or specialty. Under this assumption, BillingRate depends directly on TechnicianID and no violation exists; if the business rule changes to a company-wide flat rate, CompanyName and BillingRate would need to be extracted into a separate CONTRACTOR_COMPANY entity. No other transitive dependencies were found across the 14 tables.

3.4 Summary

The relational schema contains 14 tables, within the 10 to 20 table range required by the project guidelines. Three tables exist because of weak entities or a multivalued attribute with a composite primary key: UNIT, PAYMENT, and TECHNICIAN_SKILL. Two tables are associative entities

that resolve a many-to-many relationship while carrying their own attributes and history: LEASE and PARKING_ASSIGNMENT. One supertype and subtype split produced two additional tables, IN_HOUSE_TECH and CONTRACTOR_TECH, alongside the TECHNICIAN supertype. One additional table, MANAGER, corresponds to the entity separated from PROPERTY during conceptual design (Section 2.6); the logical design confirms this split satisfies 3NF and carries it forward into the relational schema.

4. Implementation and Application

The proposed apartment portfolio database is designed to support daily leasing, payment, parking, maintenance, inspection, and technician management activities. Based on the final relational model, the system contains 14 relations: MANAGER, PROPERTY, UNIT, TENANT, LEASE, PAYMENT, PARKING_SPOT, PARKING_ASSIGNMENT, MAINTENANCE_REQUEST, INSPECTION, TECHNICIAN, IN_HOUSE_TECH, CONTRACTOR_TECH, and TECHNICIAN_SKILL. The implementation must do more than store records; it must also preserve data integrity and enforce business rules that ordinary primary keys, foreign keys, and EER cardinalities cannot fully express.

4.1 Sample Data

Representative sample records were created for the major operational relations to demonstrate normal use of the database and to test important business rules. The examples below include a valid lease renewal, non-overlapping parking assignments, valid technician subtypes, and both unassigned and completed maintenance or inspection work.

These records are intentionally selected to test the most important operational states in the design rather than to reproduce every possible row in all 14 tables. The lease pair L001 and L002 demonstrates a valid renewal sequence, while the maintenance and inspection examples confirm that nullable technician assignments are allowed before work is completed.

Relation	Sample Record(s)	Purpose
PROPERTY / UNIT	P001, Santa Clara Apartments; Unit 102, 2BR/2BA, $2,850	UNIT is identified by the composite key (PropertyID, UnitNumber).

TENANT / LEASE	T001 Emily Chen; L001 Jan to Jun 2026; L002 Jul to Dec 2026, PriorLeaseID = L001	Shows a valid renewal with consecutive, non-overlapping date ranges.
PAYMENT	L002, Payment #1, $2,850, ACH	Shows PAYMENT identified by (LeaseID, PaymentNumber).
PARKING_ASSIGNMENT	PA001: T001 to S001, Jan to Jun; PA002: T001 to S001, Jul to Dec	Preserves parking history without overlapping use of one spot.
TECHNICIAN	TC001 Alex Smith, H; TC002 Mark Davis, C	Shows valid H/C discriminator values and matching subtypes.
MAINTENANCE / INSPECTION	R001RequestStatus='Open',TechnicianID=NULL;R002RequestStatus='Completed',TechnicianID=TC001,CompletedDateset;I001ScheduledDateset,InspectorTechID=NULL	Shows that work may be unassigned before dispatch, but completed work must contain an assignee and completion date.

Table 1. Sample data highlights.

4.2 Potential Implementation Issues and Controls

Several integrity problems may arise during daily use of the system. The most important risks and their corresponding controls are summarized below.

BR-1, BR-2, and BR-3 require application-level or transaction-safe validation because ordinary EER cardinalities and basic key constraints cannot fully represent temporal overlap and renewal-sequence rules. The design also avoids storing derived values when they can be calculated from underlying operational records. For example, parking availability is determined from current PARKING_ASSIGNMENT rows, and unit occupancy status is determined from active LEASE rows. This reduces redundancy and prevents conflicting values from being stored in different parts of the database.

Potential Issue	Control / Feature
Overlapping leases for the same unit	Before a LEASE is inserted or updated, application logic checks whether its date range overlaps any existing lease for the same PropertyID and UnitNumber (BR-1).
Overlapping parking assignments	Before a PARKING_ASSIGNMENT is saved, the application checks for conflicting date ranges for the same SpotID (BR-2).

Invalid lease renewal	PriorLeaseID must reference an earlier lease for the same tenant and unit, and the new StartDate must occur after the prior EndDate (BR-3).
Technician subtype mismatch	TechType must match exactly one subtype: H requires IN_HOUSE_TECH and C requires CONTRACTOR_TECH; a technician cannot belong to both.
Incomplete completed work	A MAINTENANCE_REQUEST or INSPECTION cannot have RequestStatus='Completed' or CompletedDate set without a technician (TechnicianID or InspectorTechID respectively).
Invalid references	Primary key and foreign key constraints prevent records from referencing nonexistent properties, units, tenants, leases, parking spots, or technicians.
Inconsistent derived values	Occupancy status, parking availability, lease term, first move-in date, and total unit count are calculated from source records rather than stored independently.
Conflicting concurrent updates	Lease and parking checks should execute within controlled transactions so validation and update occur as one reliable operation.

Table 2. Potential implementation issues and controls.

4.3 Administration and Maintenance

Effective administration is required to keep the database secure, accurate, and available. Role-based access control should limit users to the functions required by their responsibilities. Leasing staff may manage tenant and lease records, accounting staff may manage payments, maintenance staff may update maintenance requests and inspections, and database administrators should retain broader privileges for system management.

Regular database backups should be scheduled to protect against accidental deletion, data corruption, or system failure. Recovery procedures should also be tested periodically so the organization can verify that backup data can actually be restored when needed.

Routine integrity checks should identify problems such as overlapping lease periods, overlapping parking assignments, incomplete completed-work records, and technician subtype mismatches. Important changes to lease and payment records should also be logged when appropriate to provide an audit trail and improve accountability.

Finally, database structures, business rules, and administrative procedures should be documented and reviewed whenever business requirements change. Controlled schema updates and regular maintenance will help keep the database consistent with apartment management operations and provide reliable information for day-to-day reporting and decision-making.